

CSE 344 – Systems Programming

Homework #4 Report

Multi-Process & Multi-Threaded Concurrent Log Analysis Pipeline

Student Name: FATİH EMRE OĞAN

Student ID: 230104004090

Course: CSE 344 – Systems Programming

Date: April 2026

Table of Contents

Table of Contents	2
1. Introduction	3
2. System Architecture	4
3. Shared Memory Layout	6
Region A — Dispatcher Input Queue	6
Region B — Per-Level Analysis Buffers (×4).....	6
Region C — Results Area.....	6
Region D — High-Priority Buffer.....	7
4. Lock Ordering and Deadlock Prevention	8
5. Thread Lifecycle	9
Reader Thread Lifecycle.....	9
Worker Thread Lifecycle (Analyzer Process)	9
6. TLS Design	10
Why flush in the destructor, not at the barrier?.....	10
7. Dispatcher Logic	11
8. Termination Chain	12
9. Challenges & Solutions	13
Challenge 1: TLS Flush Timing	13
Challenge 2: Dispatcher Timeout with eof_count_per_level.....	13
Challenge 3: Overlapping Keyword Search.....	13
10. Test Scenarios	14
Test 1 — Baseline	14
Test 2 — Multi-keyword.....	16
Test 3 — Single Thread.....	17
Test 4 — High Concurrency (Determinism Check).....	18
Test 5 — No Priority Matches.....	20
Test 6 — Valgrind.....	21
Test 7 — Ctrl+C Handling.....	22

1. Introduction

This report documents the design and implementation of a Concurrent Log Analysis Pipeline developed for CSE 344 – System Programming, Homework #4. The system processes multiple log files simultaneously using a multi-process and multi-threaded architecture in C on a POSIX-compatible Linux environment.

The pipeline consists of seven distinct process and thread types working in concert: Reader Processes parse log files concurrently, a Dispatcher Process routes entries to level-specific buffers, four Analyzer Processes perform keyword-weighted scoring, an Aggregator Process produces structured output files, and a Watchdog Thread provides live progress reporting. All inter-process communication is performed through shared memory regions protected by process-shared POSIX primitives.

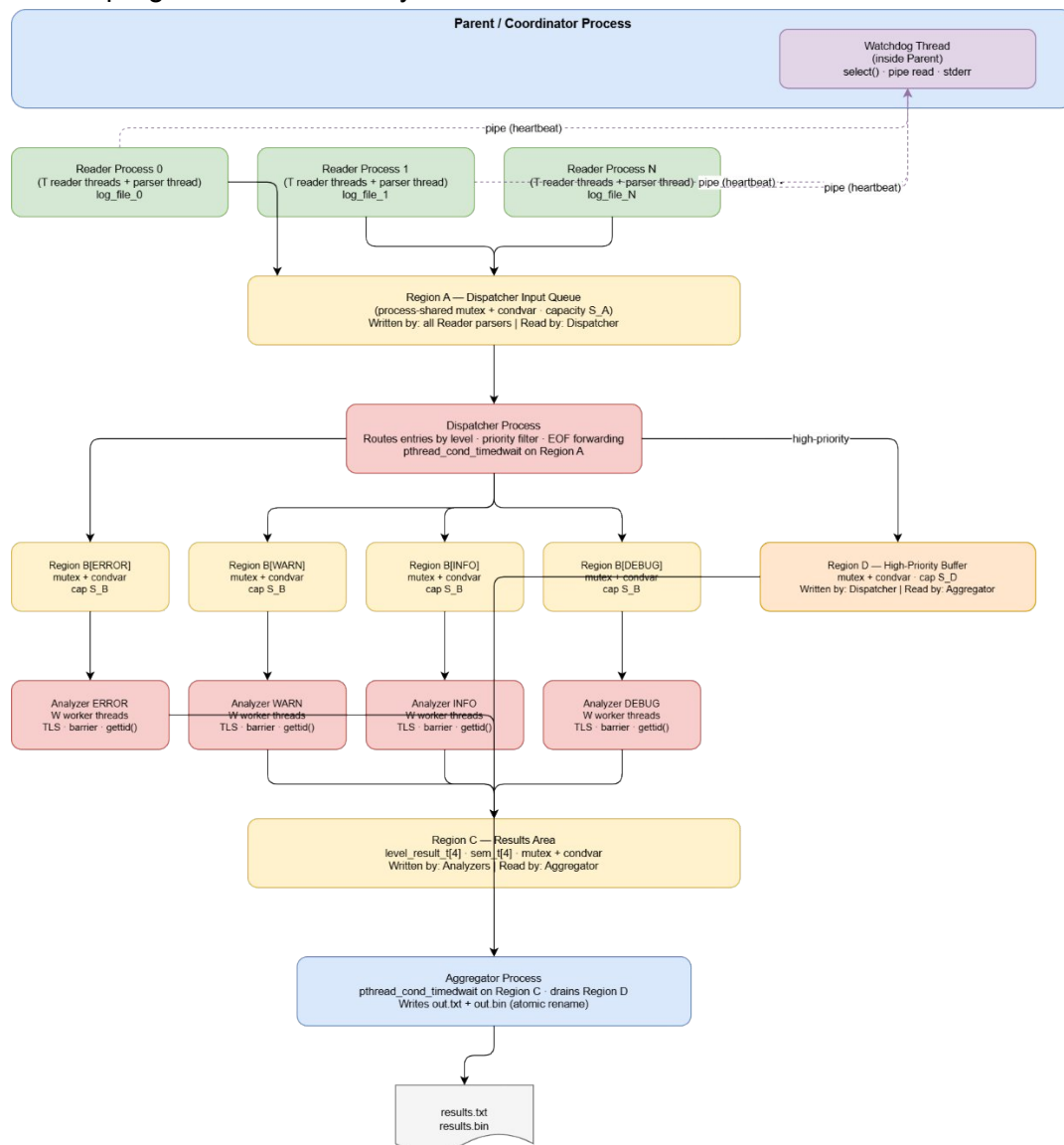
The main challenges addressed in this implementation include:

- Correctly ordering lock acquisitions across four shared memory regions to avoid deadlock.
- Using thread-local storage (`pthread_key_t`) for per-thread keyword score accumulation, with flushing deferred to the TLS destructor rather than the barrier.
- Implementing overlapping keyword search using a sliding-window algorithm.
- Handling multiple EOF markers from multiple Reader Processes in the Dispatcher without races.
- Ensuring atomic output file creation using `rename(2)` on a temporary file.
- Clean SIGINT handling with `async-signal-safe` operations only inside the signal handler.

2. System Architecture

The system comprises the following process and thread types:

- **Parent / Coordinator Process** — Initializes shared memory, forks all children, starts Watchdog Thread, waits for children.
- **Reader Process (×N)** — One per log file. Spawns T reader threads and 1 parser thread. Writes heartbeats to a dedicated pipe.
- **Dispatcher Process (×1)** — Reads from Region A, routes entries to Region B (per-level) and Region D (high-priority).
- **Analyzer Process (×4)** — One per severity level. Spawns W worker threads using TLS for score accumulation.
- **Aggregator Process (×1)** — Waits for all Region C results, writes text and binary output files.
- **Watchdog Thread** — Runs inside the parent, reads heartbeat pipes via select(), prints progress to stderr every 3 seconds.



3. Shared Memory Layout

Four shared memory regions are created by the parent using `mmap(MAP_SHARED|MAP_ANONYMOUS)` before any fork. All mutexes and condition variables use `PTHREAD_PROCESS_SHARED` attribute.

Region A — Dispatcher Input Queue

A single circular buffer of `log_entry_t` fed by all Reader Processes and consumed by the Dispatcher.

```
pthread_mutex_t  input_mutex;           // protects all Region A fields
pthread_cond_t   not_full_a;            // Readers block when buffer is full
pthread_cond_t   not_empty_a;           // Dispatcher blocks when buffer is empty
int              eof_count_per_level[4]; // incremented by each parser thread
int              total_readers;          // set by parent before fork
int              count, head, tail;      // circular buffer state
log_entry_t      buf[capacity_a];       // flexible array member
```

Region B — Per-Level Analysis Buffers (×4)

Four independent circular buffers, one per severity level (ERROR, WARN, INFO, DEBUG). Written only by the Dispatcher; read only by the corresponding Analyzer Process.

```
pthread_mutex_t  level_mutex;           // protects this level's buffer
pthread_cond_t   not_full_b;            // Dispatcher blocks when full
pthread_cond_t   not_empty_b;           // Analyzer workers block when empty
int              eof_posted;            // set to 1 by Dispatcher for this level
int              count, head, tail;
log_entry_t      buf[capacity_b];
```

Region C — Results Area

Holds one `level_result_t` per severity level, written by the reporting thread of each Analyzer Process after the barrier. Also contains four semaphores (one per level) and a process-shared condvar for the Aggregator's timed wait.

```
level_result_t   results[4];            // written by Analyzer reporting threads
pthread_mutex_t  result_mutex;          // protects results[] and results_ready
pthread_cond_t   result_cond;           // Aggregator waits here
int              results_ready;         // bitmask: bit i set when level i done
sem_t            level_sem[4];          // posted by each Analyzer on completion
```

Region D — High-Priority Buffer

A circular buffer populated by the Dispatcher with copies of entries whose SOURCE matches the priority filter. Drained by the Aggregator in a background thread.

```
pthread_mutex_t  priority_mutex;          // protects Region D
pthread_cond_t   not_full_d;              // Dispatcher blocks when full
pthread_cond_t   not_empty_d;             // Aggregator drain thread blocks when
empty
int              dispatcher_done;         // set by Dispatcher when finished
double          high_priority_score;      // written by Aggregator after drain
int              count, head, tail;
log_entry_t      buf[capacity_d];
```

4. Lock Ordering and Deadlock Prevention

A strict global lock acquisition order is enforced throughout the entire system to prevent circular dependencies:

Priority	Mutex	Purpose
1st	Region A — input_mutex	Acquired first; released before any other lock is taken
2nd	Region B — level_mutex[i]	Acquired only after releasing Region A mutex
3rd	Region C — result_mutex	Acquired only after releasing Region B mutex
4th	Region D — priority_mutex	Acquired only after releasing all other mutexes

No circular dependency exists because the ordering is strictly linear: $A \rightarrow B \rightarrow C \rightarrow D$. No code path ever acquires a lower-ranked mutex while holding a higher-ranked one. Additionally, no blocking I/O is performed while any mutex is held — all file operations and pipe writes occur exclusively outside of critical sections.

The Dispatcher is the only process that writes to both Region B and Region D. It always releases the Region A mutex before acquiring either Region B or Region D mutex, and never holds Region B and Region D simultaneously. The Aggregator only acquires Region C and Region D locks independently, never together.

5. Thread Lifecycle

Reader Thread Lifecycle

- Creation: `pthread_create()` called by the Reader Process main function. Each thread receives a byte-range [start, end) of the log file.
- Work loop: Opens the file, seeks to `range_start` (skipping partial lines if not the first thread), reads lines with `fgets()`, parses each line, and pushes valid `log_entry_t` structs into the internal bounded circular buffer using `pthread_mutex_t` + `pthread_cond_t`.
- Heartbeat: After every 50 lines processed, writes a heartbeat message to the dedicated pipe using `write(2)` (async-signal-safe).
- Termination: When the byte range is exhausted or EOF is reached, the thread decrements `ibuf.writers` under the internal mutex. If it is the last writer, it sets `ibuf.done = 1` and broadcasts on `not_empty` to wake the parser thread.
- Joined by: The Reader Process main function joins all reader threads before joining the parser thread.

Worker Thread Lifecycle (Analyzer Process)

- Creation: `pthread_create()` called by `analyzer_process_main()`. Each worker receives a `worker_arg_t` with its index and a pointer to the shared analyzer state.
- TLS setup: Immediately after start, the worker allocates a `tls_val_t` on the heap and calls `pthread_setspecific(tls_key, tv)` to register it as its thread-local value.
- Work loop: Locks Region B `level_mutex`, waits on `not_empty_b` with `pthread_cond_wait()`, pops one `log_entry_t`, releases the lock, counts overlapping keyword occurrences in the message using a sliding window, and accumulates weighted scores into the TLS array.
- Barrier: After the work loop exits (`eof_posted == 1` and `count == 0`), the thread calls `pthread_barrier_wait()`. All W workers block here until the last one arrives.
- Reporting: After the barrier, the thread with the lowest system TID (obtained via `syscall(SYS_gettid)`) is selected as the reporting thread. It waits for all TLS destructors to run (guaranteed after `pthread_join` returns) and then writes the aggregated results to Region C under `result_mutex`.
- TLS Destructor: When the thread exits (returns from the thread function), the pthread runtime calls `tls_destructor()`. This function flushes the per-keyword scores from the TLS array into the shared `per_thread_scores[][]` array under `local_mutex`, then frees the heap allocation.
- Joined by: The Analyzer Process main function joins all worker threads. Only after all joins complete are the TLS destructors guaranteed to have run.

6. TLS Design

Each Analyzer Process creates a single `pthread_key_t` (`tls_key`) with a destructor function registered at process startup:

```
pthread_key_create(&tls_key, tls_destructor);
```

Each worker thread allocates a heap-allocated `tls_val_t` structure containing a per-keyword score array and its worker index, then registers it:

```
tls_val_t *tv = calloc(1, sizeof(tls_val_t));
tv->worker_idx = wid;
tv->shared      = &sh;
pthread_setspecific(tls_key, tv);
```

During the work loop, keyword scores are accumulated exclusively into `tv->scores[]` without any locking — this is safe because TLS data is private to each thread.

The destructor function is called automatically by the pthread runtime when the thread exits:

```
static void tls_destructor(void *val) {
    tls_val_t *tv = (tls_val_t *)val;
    pthread_mutex_lock(&tv->shared->local_mutex);
    for (int k = 0; k < num_keywords; k++)
        sh->per_thread_scores[tv->worker_idx][k] += tv->scores[k];
    pthread_mutex_unlock(&tv->shared->local_mutex);
    free(val);
}
```

Why flush in the destructor, not at the barrier?

The pthread barrier guarantees that all threads have finished their work loops before any continues past the barrier. However, the TLS destructor is called after the thread function returns — after the barrier. If the flush were performed at the barrier, the reporting thread would need to read from other threads' TLS, which is impossible (TLS is private). By flushing in the destructor and joining all threads before reading the aggregated scores, the Analyzer Process main function can safely access `per_thread_scores[][]` only after all joins complete, guaranteeing all destructors have run.

7. Dispatcher Logic

The Dispatcher Process is the sole consumer of Region A and the sole producer of Region B and Region D. It runs a single loop until all EOF markers for all levels have been forwarded to Region B.

EOF marker handling across multiple Reader Processes:

- Each Reader Process has one parser thread. After all log entries are pushed to Region A, the parser thread sends one EOF marker per severity level (4 total per Reader Process). Simultaneously, it increments `ra->eof_count_per_level[lvl]` under the Region A mutex for each level.
- The Dispatcher maintains `eof_observed[4]` — a counter per level of how many EOF markers it has seen from Reader Processes. When `eof_observed[lvl]` reaches `total_readers`, the Dispatcher knows all readers have finished sending entries for that level.
- At that point, it pushes a single EOF marker for that level into Region B[lvl] and sets `eof_forwarded[lvl] = 1`. Worker threads in the Analyzer Process for that level detect `eof_posted == 1` and exit their work loops.
- The main loop exits when all four levels have `eof_forwarded` set.

Timed wait: The Dispatcher uses `pthread_cond_timedwait` on `not_empty_a` with a configurable timeout (default 10 seconds). On timeout, it checks whether `ra->eof_count_per_level[lvl] >= total_readers` for all levels. If so, it exits immediately. Otherwise it refreshes the deadline and retries. This prevents the Dispatcher from blocking indefinitely if a Reader Process crashes.

Priority routing: For each non-EOF entry, the Dispatcher checks if `entry.source` appears in the loaded priority source list. If yes, the entry is pushed to both Region B[lvl] and Region D. The single-writer constraint (only the Dispatcher writes to B and D) eliminates the need for additional inter-process coordination on these regions.

8. Termination Chain

Normal termination proceeds in the following strict order:

Step	Description
1	All reader threads in every Reader Process finish their byte ranges and signal the parser thread via <code>ibuf.done</code> .
2	Each parser thread sends 4 EOF markers (one per level) to Region A, incrementing <code>eof_count_per_level[]</code> , then exits.
3	Each Reader Process joins all its threads and calls <code>_exit(0)</code> .
4	The Dispatcher observes all EOF markers, forwards one EOF per level to Region B, sets <code>rd->dispatcher_done = 1</code> in Region D, then calls <code>_exit(0)</code> .
5	Each Analyzer Process: worker threads detect <code>eof_posted</code> in Region B, exit work loops, hit the barrier, the reporting thread writes results to Region C under <code>result_mutex</code> and broadcasts <code>result_cond</code> , posts <code>level_sem</code> , then <code>_exit(0)</code> .
6	The Aggregator wakes from <code>pthread_cond_timedwait</code> when <code>results_ready == 0xF</code> , drain thread finishes Region D, writes text and binary output files (with atomic rename), then calls <code>_exit(0)</code> .
7	The parent's <code>waitpid()</code> loop returns. The parent sets <code>shutdown_watchdog = 1</code> and calls <code>pthread_join</code> on the Watchdog Thread.
8	The Watchdog Thread detects <code>shutdown_watchdog == 1</code> after its next <code>select()</code> call and returns.
9	The parent reads final results from Region C, prints the SYSTEM SUMMARY to <code>stdout</code> , calls <code>shm_destroy()</code> to <code>munmap</code> all regions and destroy all primitives, closes all pipe file descriptors, and returns 0.

9. Challenges & Solutions

Challenge 1: TLS Flush Timing

Problem: The initial design flushed TLS scores to the shared result structure at the barrier, before thread exit. This caused a race condition because the reporting thread attempted to read `per_thread_scores[][]` while other threads had not yet flushed their TLS data — the destructor had not run yet.

Solution: The flush was moved entirely into the `pthread_key_t` destructor function. The Analyzer Process main function now reads `per_thread_scores[][]` only after `pthread_join()` returns for all worker threads, guaranteeing all destructors have completed. This required redesigning the barrier-to-exit sequence so the reporting thread does not attempt to read shared scores until after all joins.

Challenge 2: Dispatcher Timeout with `eof_count_per_level`

Problem: The Dispatcher's `pthread_cond_timedwait` timeout fallback checked `ra->eof_count_per_level[]` to determine if all readers had finished. However, `eof_count_per_level` was never incremented — only the `is_eof` flag in the entry struct was used — so the timeout exit condition could never trigger.

Solution: The parser thread in each Reader Process was modified to increment `ra->eof_count_per_level[lvl]++` under the Region A mutex immediately after pushing each EOF marker. This ensures the Dispatcher can correctly detect completion via the timeout path when a Reader Process finishes abnormally or when the main buffer is drained under timeout conditions.

Challenge 3: Overlapping Keyword Search

Problem: Using `strstr()` and advancing by the length of the keyword after each match misses overlapping occurrences. For example, searching for 'error' in 'errorerror' with `strstr()`-based advancement finds only 1 match instead of 2.

Solution: A manual sliding-window approach was implemented: the search pointer advances by exactly 1 character after each match (whether or not a match was found), not by the keyword length. This correctly identifies all overlapping occurrences at the cost of $O(n*m)$ time, which is acceptable for log message lengths.

```
while (*p) {
    if (strncmp(p, keyword, klen) == 0) { count++; }
    p++; // always advance by 1, not by klen
}
```

10. Test Scenarios

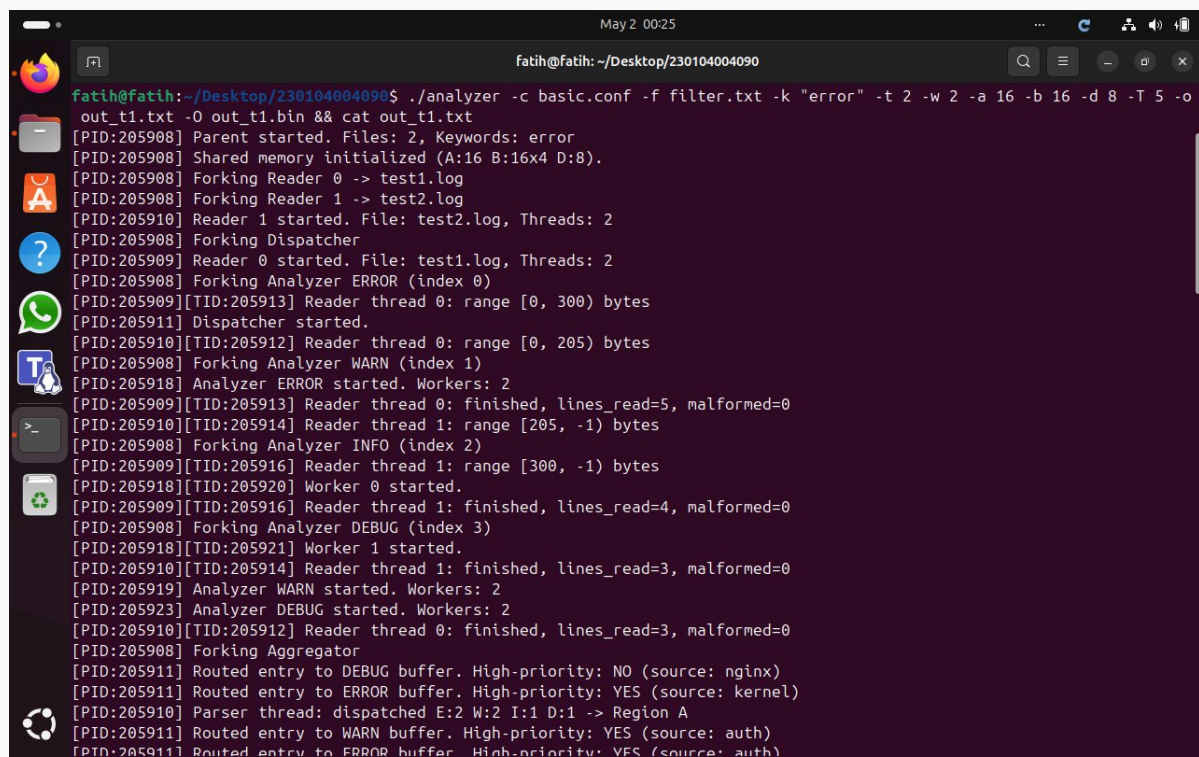
All tests were run on Debian 12 (64-bit) in VirtualBox. Screenshots below show the complete terminal output from command prompt through program exit.

Test 1 — Baseline

Verifies correct entry counts, human-readable output format, and binary file magic number 0xC5E3440B.

```
make
```

```
./analyzer -c basic.conf -f filter.txt -k "error" -t 2 -w 2 -a 16 -b 16 -d 8 -T 5  
-o out_t1.txt -O out_t1.bin && cat out_t1.txt
```



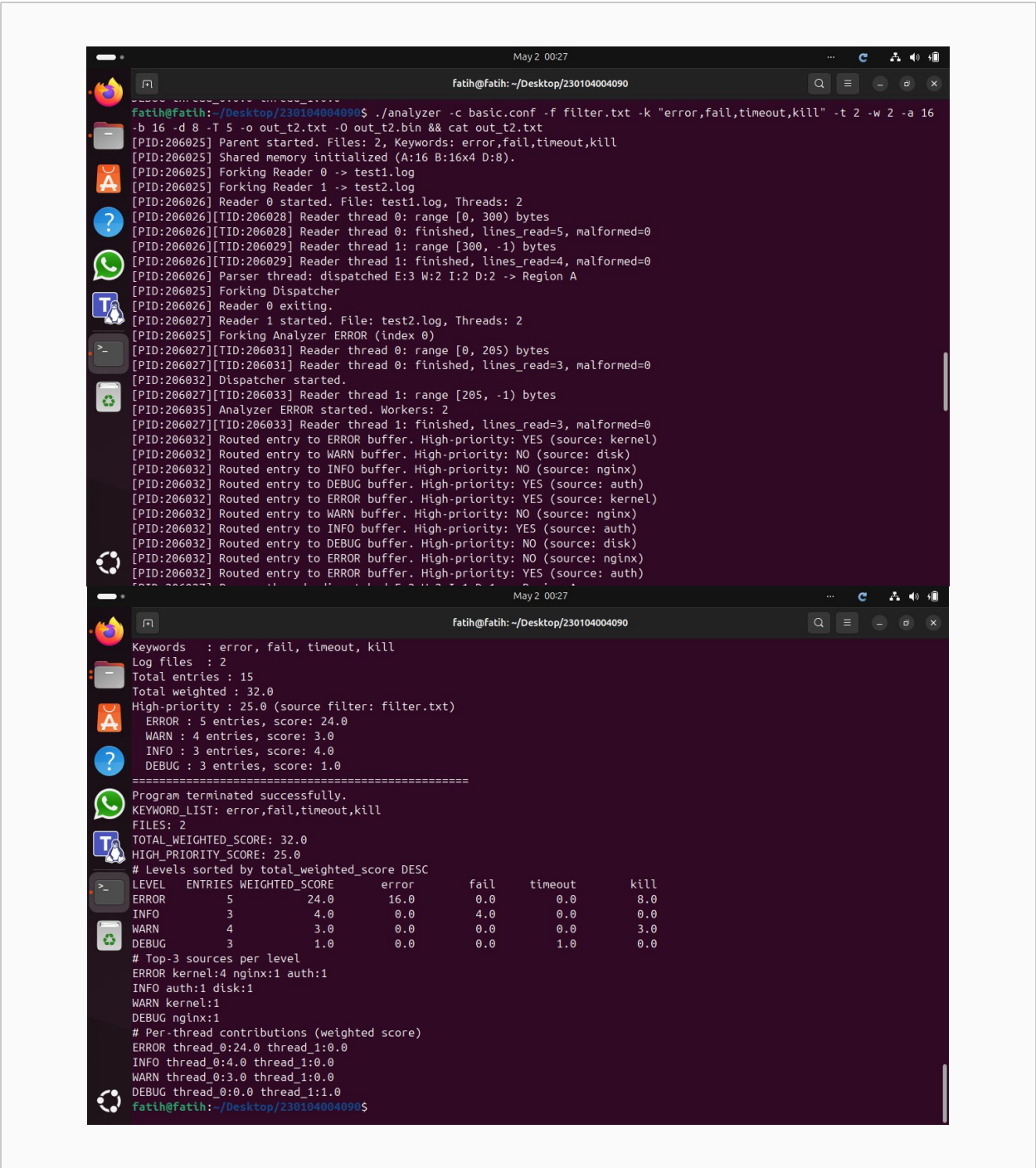
```
May 2 00:26
fatih@fatih: ~/Desktop/230104004090

SYSTEM SUMMARY
Keywords : error
Log files : 2
Total entries : 15
Total weighted : 16.0
High-priority : 12.0 (source filter: filter.txt)
ERROR : 5 entries, score: 16.0
WARN : 4 entries, score: 0.0
INFO : 3 entries, score: 0.0
DEBUG : 3 entries, score: 0.0
=====
Program terminated successfully.
KEYWORD_LIST: error
FILES: 2
TOTAL_WEIGHTED_SCORE: 16.0
HIGH_PRIORITY_SCORE: 12.0
# Levels sorted by total_weighted_score DESC
LEVEL ENTRIES WEIGHTED_SCORE error
ERROR 5 16.0 16.0
WARN 4 0.0 0.0
INFO 3 0.0 0.0
DEBUG 3 0.0 0.0
# Top-3 sources per level
ERROR kernel:2 auth:1 nginx:1
WARN
INFO
DEBUG
# Per-thread contributions (weighted score)
ERROR thread_0:8.0 thread_1:8.0
WARN thread_0:0.0 thread_1:0.0
INFO thread_0:0.0 thread_1:0.0
DEBUG thread_0:0.0 thread_1:0.0
fatih@fatih:~/Desktop/230104004090$
```

Test 2 — Multi-keyword

Verifies per-keyword score breakdown and weighted scoring across four keywords.

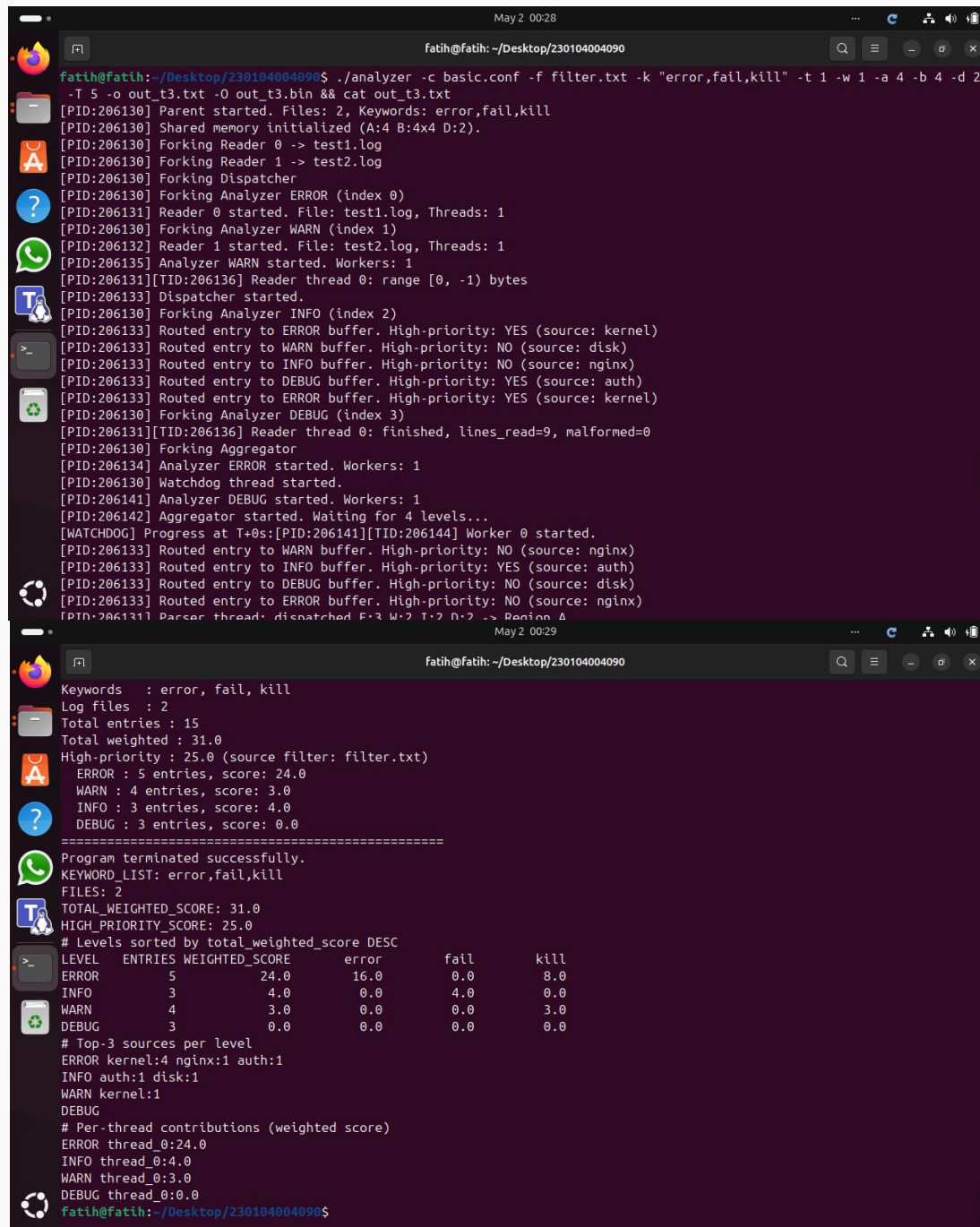
```
./analyzer -c basic.conf -f filter.txt -k "error,fail,timeout,kill" -t 2 -w 2 -a 16 -b 16 -d 8 -T 5 -o out_t2.txt -O out_t2.bin && cat out_t2.txt
```



Test 3 — Single Thread

Verifies sequential correctness with $t=1$ $w=1$. Totals must match Test 1 baseline.

```
./analyzer -c basic.conf -f filter.txt -k "error,fail,kill" -t 1 -w 1 -a 4 -b 4 -d 2 -T 5 -o out_t3.txt -O out_t3.bin && cat out_t3.txt
```



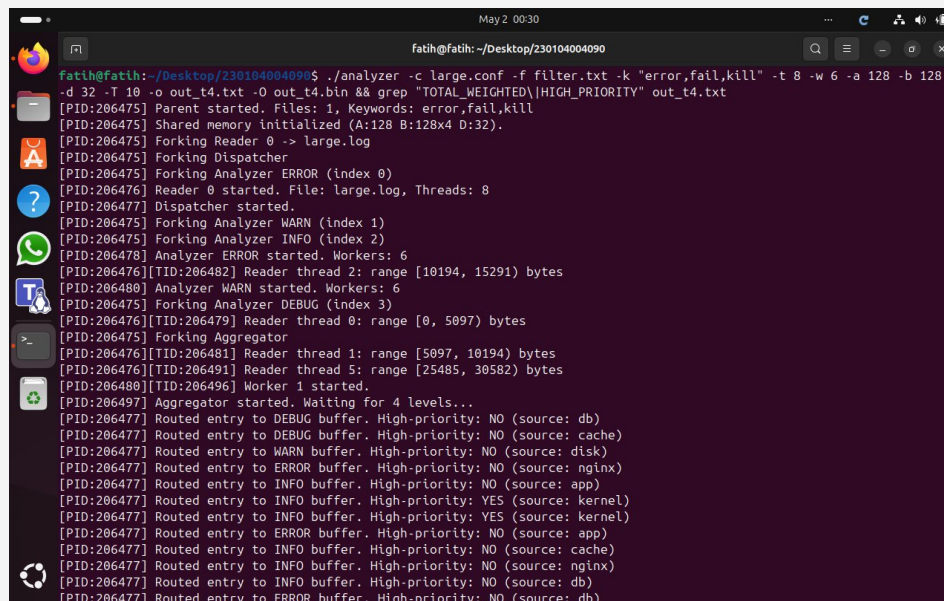
```
fatih@fatih: ~/Desktop/230104004090
fatih@fatih:~/Desktop/230104004090$ ./analyzer -c basic.conf -f filter.txt -k "error,fail,kill" -t 1 -w 1 -a 4 -b 4 -d 2 -T 5 -o out_t3.txt -O out_t3.bin && cat out_t3.txt
[PID:206130] Parent started. Files: 2, Keywords: error,fail,kill
[PID:206130] Shared memory initialized (A:4 B:4x4 D:2).
[PID:206130] Forking Reader 0 -> test1.log
[PID:206130] Forking Reader 1 -> test2.log
[PID:206130] Forking Dispatcher
[PID:206130] Forking Analyzer ERROR (index 0)
[PID:206131] Reader 0 started. File: test1.log, Threads: 1
[PID:206130] Forking Analyzer WARN (index 1)
[PID:206132] Reader 1 started. File: test2.log, Threads: 1
[PID:206135] Analyzer WARN started. Workers: 1
[PID:206131][TID:206136] Reader thread 0: range [0, -1) bytes
[PID:206133] Dispatcher started.
[PID:206130] Forking Analyzer INFO (index 2)
[PID:206133] Routed entry to ERROR buffer. High-priority: YES (source: kernel)
[PID:206133] Routed entry to WARN buffer. High-priority: NO (source: disk)
[PID:206133] Routed entry to INFO buffer. High-priority: NO (source: nginx)
[PID:206133] Routed entry to DEBUG buffer. High-priority: YES (source: auth)
[PID:206133] Routed entry to ERROR buffer. High-priority: YES (source: kernel)
[PID:206130] Forking Analyzer DEBUG (index 3)
[PID:206131][TID:206136] Reader thread 0: finished, lines_read=9, malformed=0
[PID:206130] Forking Aggregator
[PID:206134] Analyzer ERROR started. Workers: 1
[PID:206130] Watchdog thread started.
[PID:206141] Analyzer DEBUG started. Workers: 1
[PID:206142] Aggregator started. Waiting for 4 levels...
[WATCHDOG] Progress at T=0s:[PID:206141][TID:206144] Worker 0 started.
[PID:206133] Routed entry to WARN buffer. High-priority: NO (source: nginx)
[PID:206133] Routed entry to INFO buffer. High-priority: YES (source: auth)
[PID:206133] Routed entry to DEBUG buffer. High-priority: NO (source: disk)
[PID:206133] Routed entry to ERROR buffer. High-priority: NO (source: nginx)
[PID:206131] Parser thread: dispatched F:3 W:2 T:2 D:2 -> Deninn A
May2 00:29

Keywords : error, fail, kill
Log files : 2
Total entries : 15
Total weighted : 31.0
High-priority : 25.0 (source filter: filter.txt)
  ERROR : 5 entries, score: 24.0
  WARN : 4 entries, score: 3.0
  INFO : 3 entries, score: 4.0
  DEBUG : 3 entries, score: 0.0
=====
Program terminated successfully.
KEYWORD_LIST: error,fail,kill
FILES: 2
TOTAL_WEIGHTED_SCORE: 31.0
HIGH_PRIORITY_SCORE: 25.0
# Levels sorted by total_weighted_score DESC
LEVEL  ENTRIES WEIGHTED_SCORE  error  fail  kill
ERROR   5         24.0         16.0   0.0   8.0
INFO    3          4.0          0.0   4.0   0.0
WARN    4          3.0          0.0   0.0   3.0
DEBUG   3          0.0          0.0   0.0   0.0
# Top-3 sources per level
ERROR kernel:4 nginx:1 auth:1
INFO  auth:1 disk:1
WARN  kernel:1
DEBUG
# Per-thread contributions (weighted score)
ERROR thread_0:24.0
INFO  thread_0:4.0
WARN  thread_0:3.0
DEBUG thread_0:0.0
fatih@fatih:~/Desktop/230104004090$
```


Test 4 — High Concurrency (Determinism Check)

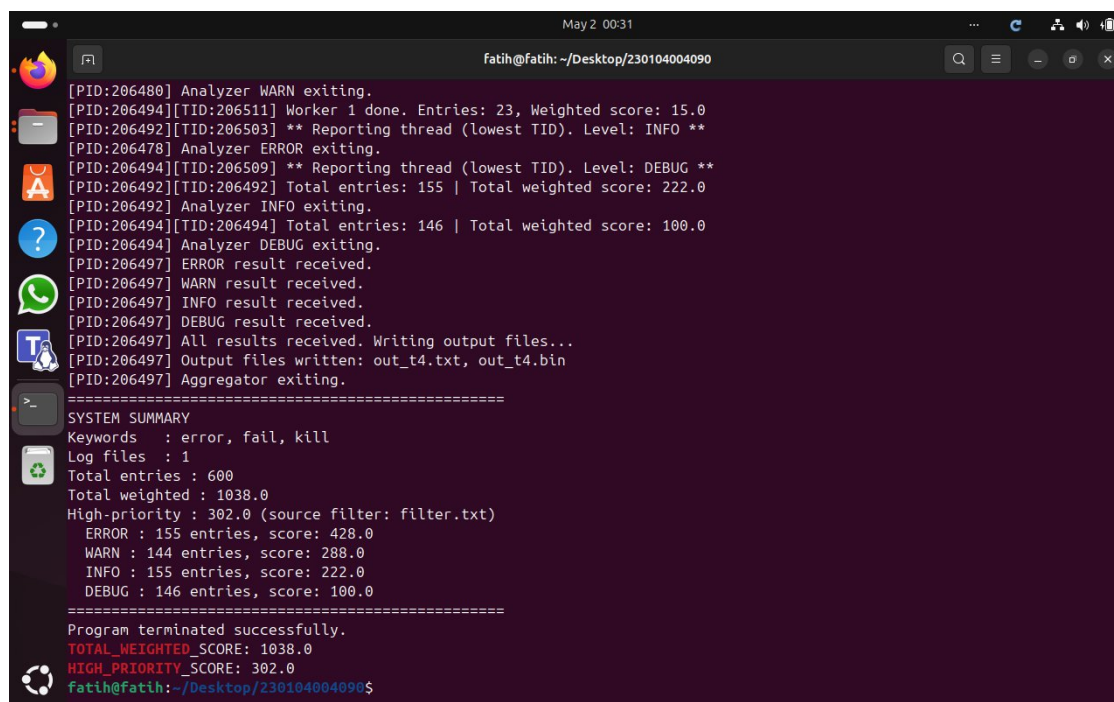
Runs 3 times with t=8 w=6 on 600+ line log file. TOTAL_WEIGHTED_SCORE must be identical across all runs.

```
./analyzer -c large.conf -f filter.txt -k "error,fail,kill" -t 8 -w 6 -a 128 -b 128 -d 32 -T 10 -o out_t4.txt -O out_t4.bin && grep "TOTAL_WEIGHTED" out_t4.txt
```



```
fatih@fatih: ~/Desktop/230104004090
fatih@fatih:~/Desktop/230104004090$ ./analyzer -c large.conf -f filter.txt -k "error,fail,kill" -t 8 -w 6 -a 128 -b 128 -d 32 -T 10 -o out_t4.txt -O out_t4.bin && grep "TOTAL_WEIGHTED" out_t4.txt
[PID:206475] Parent started. Files: 1, Keywords: error,fail,kill
[PID:206475] Shared memory initialized (A:128 B:128x4 D:32).
[PID:206475] Forking Reader 0 -> large.log
[PID:206475] Forking Dispatcher
[PID:206475] Forking Analyzer ERROR (index 0)
[PID:206476] Reader 0 started. File: large.log, Threads: 8
[PID:206477] Dispatcher started.
[PID:206475] Forking Analyzer WARN (index 1)
[PID:206475] Forking Analyzer INFO (index 2)
[PID:206478] Analyzer ERROR started. Workers: 6
[PID:206476][TID:206482] Reader thread 2: range [10194, 15291] bytes
[PID:206480] Analyzer WARN started. Workers: 6
[PID:206475] Forking Analyzer DEBUG (index 3)
[PID:206476][TID:206479] Reader thread 0: range [0, 5097] bytes
[PID:206475] Forking Aggregator
[PID:206476][TID:206481] Reader thread 1: range [5097, 10194] bytes
[PID:206476][TID:206491] Reader thread 5: range [25485, 30582] bytes
[PID:206480][TID:206496] Worker 1 started.
[PID:206497] Aggregator started. Waiting for 4 levels...
[PID:206477] Routed entry to DEBUG buffer. High-priority: NO (source: db)
[PID:206477] Routed entry to DEBUG buffer. High-priority: NO (source: cache)
[PID:206477] Routed entry to WARN buffer. High-priority: NO (source: disk)
[PID:206477] Routed entry to ERROR buffer. High-priority: NO (source: nginx)
[PID:206477] Routed entry to INFO buffer. High-priority: NO (source: app)
[PID:206477] Routed entry to INFO buffer. High-priority: YES (source: kernel)
[PID:206477] Routed entry to INFO buffer. High-priority: YES (source: kernel)
[PID:206477] Routed entry to ERROR buffer. High-priority: NO (source: app)
[PID:206477] Routed entry to INFO buffer. High-priority: NO (source: cache)
[PID:206477] Routed entry to INFO buffer. High-priority: NO (source: nginx)
[PID:206477] Routed entry to INFO buffer. High-priority: NO (source: db)
[PID:206477] Routed entry to ERROR buffer. High-priority: NO (source: db)
```

Run 1 — TOTAL_WEIGHTED_SCORE: 1038.0



```
fatih@fatih: ~/Desktop/230104004090
[PID:206480] Analyzer WARN exiting.
[PID:206494][TID:206511] Worker 1 done. Entries: 23, Weighted score: 15.0
[PID:206492][TID:206503] ** Reporting thread (lowest TID). Level: INFO **
[PID:206478] Analyzer ERROR exiting.
[PID:206494][TID:206509] ** Reporting thread (lowest TID). Level: DEBUG **
[PID:206492][TID:206492] Total entries: 155 | Total weighted score: 222.0
[PID:206492] Analyzer INFO exiting.
[PID:206494][TID:206494] Total entries: 146 | Total weighted score: 100.0
[PID:206494] Analyzer DEBUG exiting.
[PID:206497] ERROR result received.
[PID:206497] WARN result received.
[PID:206497] INFO result received.
[PID:206497] DEBUG result received.
[PID:206497] All results received. Writing output files...
[PID:206497] Output files written: out_t4.txt, out_t4.bin
[PID:206497] Aggregator exiting.
=====
SYSTEM SUMMARY
Keywords   : error, fail, kill
Log files  : 1
Total entries : 600
Total weighted : 1038.0
High-priority : 302.0 (source filter: filter.txt)
  ERROR : 155 entries, score: 428.0
  WARN  : 144 entries, score: 288.0
  INFO  : 155 entries, score: 222.0
  DEBUG : 146 entries, score: 100.0
=====
Program terminated successfully.
TOTAL_WEIGHTED_SCORE: 1038.0
HIGH_PRIORITY_SCORE: 302.0
fatih@fatih:~/Desktop/230104004090$
```

Run 2 — TOTAL_WEIGHTED_SCORE: 1038.0

```
fatih@fatih: ~/Desktop/230104004090
[PID:206626][TID:206635] Worker 2 done. Entries: 26, Weighted score: 45.0
[PID:206621][TID:206621] Total entries: 155 | Total weighted score: 428.0
[PID:206621] Analyzer ERROR exiting.
[PID:206626][TID:206633] ** Reporting thread (lowest TID). Level: WARN **
[PID:206626][TID:206626] Total entries: 144 | Total weighted score: 288.0
[PID:206626] Analyzer WARN exiting.
[PID:206644][TID:206649] ** Reporting thread (lowest TID). Level: DEBUG **
[PID:206644][TID:206644] Total entries: 146 | Total weighted score: 100.0
[PID:206644] Analyzer DEBUG exiting.
[PID:206646] ERROR result received.
[PID:206646] WARN result received.
[PID:206646] INFO result received.
[PID:206646] DEBUG result received.
[PID:206646] All results received. Writing output files...
[PID:206646] Output files written: out_t4.txt, out_t4.bin
[PID:206646] Aggregator exiting.
=====
SYSTEM SUMMARY
Keywords   : error, fail, kill
Log files  : 1
Total entries : 600
Total weighted : 1038.0
High-priority : 302.0 (source filter: filter.txt)
  ERROR : 155 entries, score: 428.0
  WARN  : 144 entries, score: 288.0
  INFO  : 155 entries, score: 222.0
  DEBUG : 146 entries, score: 100.0
=====
Program terminated successfully.
TOTAL_WEIGHTED_SCORE: 1038.0
HIGH_PRIORITY_SCORE: 302.0
fatih@fatih:~/Desktop/230104004090$
```

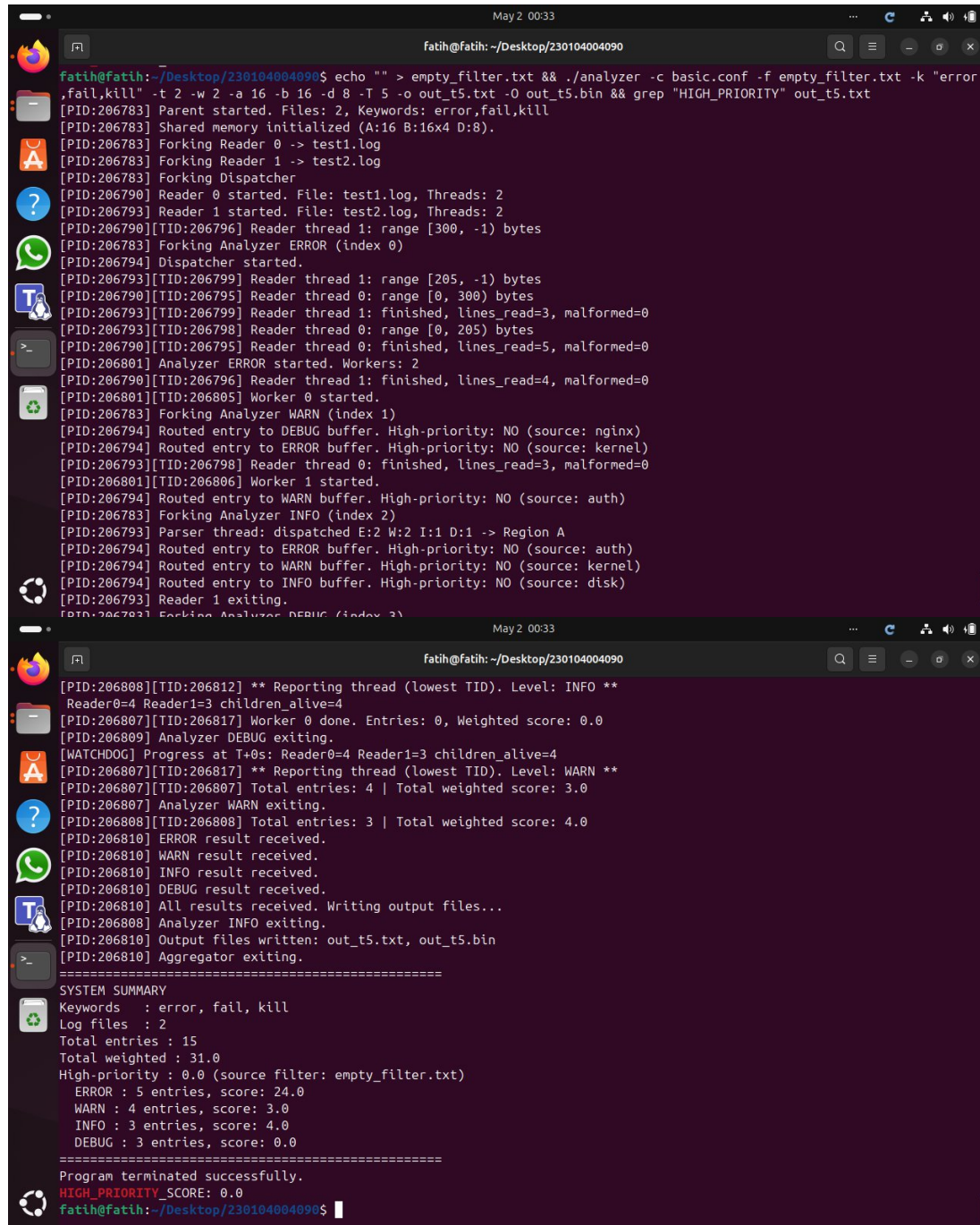
Run 3 — TOTAL_WEIGHTED_SCORE: 1038.0

```
fatih@fatih: ~/Desktop/230104004090
[PID:206700] Analyzer WARN exiting.
[PID:206711][TID:206718] ** Reporting thread (lowest TID). Level: DEBUG **
[PID:206703][TID:206719] Worker 2 done. Entries: 21, Weighted score: 26.0
[PID:206703][TID:206736] Worker 5 done. Entries: 28, Weighted score: 36.0
[PID:206703][TID:206716] ** Reporting thread (lowest TID). Level: INFO **
[PID:206711][TID:206711] Total entries: 146 | Total weighted score: 100.0
[PID:206711] Analyzer DEBUG exiting.
[PID:206703][TID:206703] Total entries: 155 | Total weighted score: 222.0
[PID:206703] Analyzer INFO exiting.
[PID:206712] ERROR result received.
[PID:206712] WARN result received.
[PID:206712] INFO result received.
[PID:206712] DEBUG result received.
[PID:206712] All results received. Writing output files...
[PID:206712] Output files written: out_t4.txt, out_t4.bin
[PID:206712] Aggregator exiting.
=====
SYSTEM SUMMARY
Keywords   : error, fail, kill
Log files  : 1
Total entries : 600
Total weighted : 1038.0
High-priority : 302.0 (source filter: filter.txt)
  ERROR : 155 entries, score: 428.0
  WARN  : 144 entries, score: 288.0
  INFO  : 155 entries, score: 222.0
  DEBUG : 146 entries, score: 100.0
=====
Program terminated successfully.
TOTAL_WEIGHTED_SCORE: 1038.0
HIGH_PRIORITY_SCORE: 302.0
fatih@fatih:~/Desktop/230104004090$
```

Test 5 — No Priority Matches

Uses an empty filter file. HIGH_PRIORITY_SCORE must be 0.0 and the program must not crash.

```
echo "" > empty_filter.txt && ./analyzer -c basic.conf -f empty_filter.txt -k "error, fail, kill" -t 2 -w 2 -a 16 -b 16 -d 8 -T 5 -o out_t5.txt -O out_t5.bin && grep "HIGH_PRIORITY" out_t5.txt
```

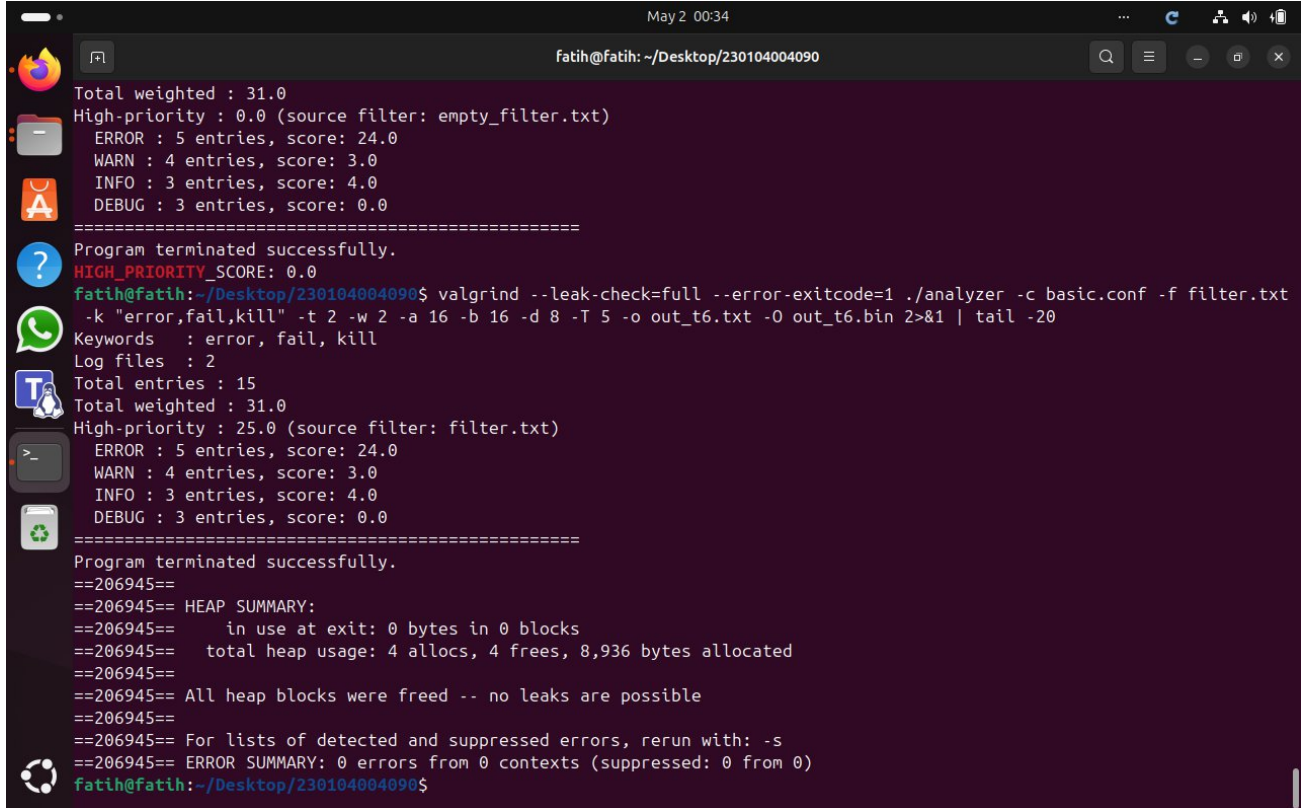


```
fatih@fatih: ~/Desktop/230104004090
fatih@fatih:~/Desktop/230104004090$ echo "" > empty_filter.txt && ./analyzer -c basic.conf -f empty_filter.txt -k "error, fail, kill" -t 2 -w 2 -a 16 -b 16 -d 8 -T 5 -o out_t5.txt -O out_t5.bin && grep "HIGH_PRIORITY" out_t5.txt
[PID:206783] Parent started. Files: 2, Keywords: error, fail, kill
[PID:206783] Shared memory initialized (A:16 B:16x4 D:8).
[PID:206783] Forking Reader 0 -> test1.log
[PID:206783] Forking Reader 1 -> test2.log
[PID:206783] Forking Dispatcher
[PID:206790] Reader 0 started. File: test1.log, Threads: 2
[PID:206793] Reader 1 started. File: test2.log, Threads: 2
[PID:206790][TID:206796] Reader thread 1: range [300, -1) bytes
[PID:206783] Forking Analyzer ERROR (index 0)
[PID:206794] Dispatcher started.
[PID:206793][TID:206799] Reader thread 1: range [205, -1) bytes
[PID:206790][TID:206795] Reader thread 0: range [0, 300) bytes
[PID:206793][TID:206799] Reader thread 1: finished, lines_read=3, malformed=0
[PID:206793][TID:206798] Reader thread 0: range [0, 205) bytes
[PID:206790][TID:206795] Reader thread 0: finished, lines_read=5, malformed=0
[PID:206801] Analyzer ERROR started. Workers: 2
[PID:206790][TID:206796] Reader thread 1: finished, lines_read=4, malformed=0
[PID:206801][TID:206805] Worker 0 started.
[PID:206783] Forking Analyzer WARN (index 1)
[PID:206794] Routed entry to DEBUG buffer. High-priority: NO (source: nginx)
[PID:206794] Routed entry to ERROR buffer. High-priority: NO (source: kernel)
[PID:206793][TID:206798] Reader thread 0: finished, lines_read=3, malformed=0
[PID:206801][TID:206806] Worker 1 started.
[PID:206794] Routed entry to WARN buffer. High-priority: NO (source: auth)
[PID:206783] Forking Analyzer INFO (index 2)
[PID:206793] Parser thread: dispatched E:2 W:2 I:1 D:1 -> Region A
[PID:206794] Routed entry to ERROR buffer. High-priority: NO (source: auth)
[PID:206794] Routed entry to WARN buffer. High-priority: NO (source: kernel)
[PID:206794] Routed entry to INFO buffer. High-priority: NO (source: disk)
[PID:206793] Reader 1 exiting.
[PID:206793] Forking Analyzer DEBUG (index 3)
[PID:206808][TID:206812] ** Reporting thread (lowest TID). Level: INFO **
Reader0=4 Reader1=3 children_alive=4
[PID:206807][TID:206817] Worker 0 done. Entries: 0, Weighted score: 0.0
[PID:206809] Analyzer DEBUG exiting.
[WATCHDOG] Progress at T+0s: Reader0=4 Reader1=3 children_alive=4
[PID:206807][TID:206817] ** Reporting thread (lowest TID). Level: WARN **
[PID:206807][TID:206807] Total entries: 4 | Total weighted score: 3.0
[PID:206807] Analyzer WARN exiting.
[PID:206808][TID:206808] Total entries: 3 | Total weighted score: 4.0
[PID:206810] ERROR result received.
[PID:206810] WARN result received.
[PID:206810] INFO result received.
[PID:206810] DEBUG result received.
[PID:206810] All results received. Writing output files...
[PID:206808] Analyzer INFO exiting.
[PID:206810] Output files written: out_t5.txt, out_t5.bin
[PID:206810] Aggregator exiting.
=====
SYSTEM SUMMARY
Keywords : error, fail, kill
Log files : 2
Total entries : 15
Total weighted : 31.0
High-priority : 0.0 (source filter: empty_filter.txt)
ERROR : 5 entries, score: 24.0
WARN : 4 entries, score: 3.0
INFO : 3 entries, score: 4.0
DEBUG : 3 entries, score: 0.0
=====
Program terminated successfully.
HIGH_PRIORITY_SCORE: 0.0
fatih@fatih:~/Desktop/230104004090$
```


Test 6 — Valgrind

Checks for memory leaks and errors. Expected: 0 errors, 0 leaks, all heap blocks freed.

```
valgrind --leak-check=full --error-exitcode=1 ./analyzer -c basic.conf -f
filter.txt -k "error,fail,kill" -t 2 -w 2 -a 16 -b 16 -d 8 -T 5 -o out_t6.txt -O
out_t6.bin 2>&1 | tail -20
```



```
May 2 00:34
fatih@fatih: ~/Desktop/230104004090
Total weighted : 31.0
High-priority : 0.0 (source filter: empty_filter.txt)
ERROR : 5 entries, score: 24.0
WARN : 4 entries, score: 3.0
INFO : 3 entries, score: 4.0
DEBUG : 3 entries, score: 0.0
=====
Program terminated successfully.
HIGH_PRIORITY_SCORE: 0.0
fatih@fatih:~/Desktop/230104004090$ valgrind --leak-check=full --error-exitcode=1 ./analyzer -c basic.conf -f filter.txt
-k "error,fail,kill" -t 2 -w 2 -a 16 -b 16 -d 8 -T 5 -o out_t6.txt -O out_t6.bin 2>&1 | tail -20
Keywords : error, fail, kill
Log files : 2
Total entries : 15
Total weighted : 31.0
High-priority : 25.0 (source filter: filter.txt)
ERROR : 5 entries, score: 24.0
WARN : 4 entries, score: 3.0
INFO : 3 entries, score: 4.0
DEBUG : 3 entries, score: 0.0
=====
Program terminated successfully.
==206945==
==206945== HEAP SUMMARY:
==206945==    in use at exit: 0 bytes in 0 blocks
==206945==    total heap usage: 4 allocs, 4 frees, 8,936 bytes allocated
==206945==
==206945== All heap blocks were freed -- no leaks are possible
==206945==
==206945== For lists of detected and suppressed errors, rerun with: -s
==206945== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
fatih@fatih:~/Desktop/230104004090$
```

Test 7 — Ctrl+C Handling

Verifies no zombie processes and no orphan shared memory segments after SIGINT.

```
./analyzer -c large.conf -f filter.txt -k "error,fail,kill" -t 1 -w 1 -a 4 -b 4 -d 2 -T 30 -o out_t7.txt -O out_t7.bin &
```

```
sleep 2 && kill -INT $! && sleep 6 && ps aux | grep defunct | grep -v grep || echo "No zombies" && ipcs -m
```

The screenshot shows a terminal window with the following content:

```
fatih@fatih: ~/Desktop/230104004090
fatih@fatih:~/Desktop/230104004090$ ./analyzer -c huge2.conf -f filter.txt -k "error,fail,kill" -t 1 -w 1 -a 4 -b 4 -d 2 -T 30 -o out_t7.txt -O out_t7.bin
```

The terminal output shows the analyzer running and processing data. It includes a "SYSTEM SUMMARY" section:

```
SYSTEM SUMMARY
Keywords : error, fail, kill
Log files : 1
Total entries : 600
Total weighted : 1038.0
High-priority : 302.0 (source filter: filter.txt)
  ERROR : 155 entries, score: 428.0
  WARN : 144 entries, score: 288.0
  INFO : 155 entries, score: 222.0
  DEBUG : 146 entries, score: 100.0
```

The program terminates successfully. The user then runs the verification command:

```
fatih@fatih:~/Desktop/230104004090$ ps aux | grep defunct | grep -v grep || echo "No zombies" && ipcs -m | grep -v "key"
```

The output of the verification command is:

```
fatih 10846  0.0  0.0  0  0 ?        Z   Apr30   0:00 [sd_espeak-ng-mb] <defunct>
```